

Sistemi informativi su Web

Gestione Persistenza: repository aa 2023-2024



Paolo Merialdo
Università degli Studi Roma Tre



This work is licensed under a Creative Commons Attribution-NonCommercial-Share Alike 3.0 United States
See <http://creativecommons.org/licenses/by-nc-sa/3.0/us/> for details

Sommario

- Repository
- JPA CrudRepository
 - Il metodo save()
 - Costruzione di query
 - Custom query
- Eventi in cascata

Repository

- Il pattern Repository viene usato negli approcci Domain Driven Design (DDD)
- I repository sono oggetti che incapsulano la logica necessaria per l'accesso al database
- Favoriscono la manutenibilità del codice disaccoppiando la tecnologia utilizzata per accedere al database dal modello di dominio

Il pattern Repository

- Martin Fowler describe il pattern repository così:
 - *A repository performs the tasks of an **intermediary between the domain model layers and data mapping**, acting in a similar way to a set of domain objects in memory*
 - *Conceptually, a repository encapsulates a set of objects stored in the database and operations that can be performed on them, providing a way that is closer to the persistence layer*
 - *Repositories, also, support the purpose of separating, clearly and in one direction, the dependency between the work domain and the data allocation or mapping*

Il pattern Repository

- Possiamo pensare un repository come ad una collezione persistente
 - Il programmatore/utilizzatore usa i repository senza che gli sia richiesto di conoscere i dettagli di come internamente viene gestita la persistenza
- Definiamo una classe Repository per (la radice di) ogni aggregato del modello di dominio
- Il repository offre metodi per salvare, recuperare, cancellare oggetti dalla collezione, più altre funzioni di utilità (ad es., conteggio, verifica di esistenza)

Il pattern Repository

- I metodi delle classi repository non devono essere transazionali, la transazione deve essere aperta (e chiusa) dall'oggetto che invoca il metodo del repository
- Infatti, le operazioni offerte dal sistema (che usano i metodi dei repository) possono essere composte da più operazioni fornite da repository diversi: la transazione deve poter essere aperta e chiusa da chi usa i repository

Spring Boot: Persistenza

- Il framework offre un sistema di librerie per la definizione dei Repository
- Per scrivere una classe Repository è sufficiente definire una Java interface che estende
`org.springframework.data.repository.CrudRepository`
- Automaticamente avremo a disposizione i metodi crud per la nostra classe. Vedi:

<https://docs.spring.io/spring-data/commons/docs/current/api/org/springframework/data/repository/CrudRepository.html>

CrudRepository

long	<u>count()</u> Returns the number of entities available.
void	<u>delete(T entity)</u> Deletes a given entity.
void	<u>deleteAll()</u> Deletes all entities managed by the repository.
void	<u>deleteAll(Iterable<? extends T> entities)</u> Deletes the given entities.
void	<u>deleteAllById(Iterable<? extends ID> ids)</u> Deletes all instances of the type T with the given IDs.
void	<u>deleteById(ID id)</u> Deletes the entity with the given id.
boolean	<u>existsById(ID id)</u> Returns whether an entity with the given id exists.
<u>Iterable<T></u>	<u>findAll()</u> Returns all instances of the type.
<u>Iterable<T></u>	<u>findAllById(Iterable<ID> ids)</u> Returns all instances of the type T with the given IDs.
<u>Optional<T></u>	<u>findById(ID id)</u> Retrieves an entity by its id.
<S extends <u>T</u> > S	<u>save(S entity)</u> Saves a given entity.
<S extends <u>T</u> > <u>Iterable<S></u>	<u>saveAll(Iterable<S> entities)</u> Saves all given entities.

CrudRepository: Esempio

```
package it.uniroma3.siw.spring.repository;
```

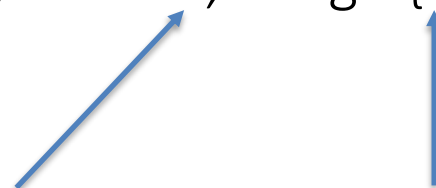
```
import java.util.List;
```

```
import org.springframework.data.repository CrudRepository;
```

```
import it.uniroma3.siw.spring.model.Movie;
```

```
public interface MovieRepository extends CrudRepository<Movie, Long> {  
}
```

Classe dell'entità



Classe dell'id

Il metodo save(S entity)

<S extends I> S save(S entity)

- Se il valore della variabile annotata @Id è nullo, viene salvata nel database una ennupla, con creazione della chiave.
L'oggetto passato come parametro viene restituito con il valore dell'id assegnato dal database
- Se l'entità esiste nel database (il valore della variabile annotata @Id è diverso da null), la ennupla corrispondente viene aggiornata nel database

Esempio

Mi permette
di eseguire
l'app

```
@SpringBootApplication
public class SiwJpa01Application implements CommandLineRunner {

    @Autowired
    private ProductRepository productRepository;

    public static void main(String[] args) {
        SpringApplication.run(SiwJpa01Application.class, args);
    }

    @Override
    public void run(String... args) throws Exception {

        Product product = new Product();
        product.setCode("abc");
        product.setName("penna");
        product.setPrice(2.1f);
        System.out.println("prima di save: " + product);
        productRepository.save(product);
        System.out.println("dopo save: " + product);
    }
}
```

CrudRepository: costruzione di query

- All'interfaccia è possibile aggiungere metodi *find*, *delete*, *count*, *exists* specifici con una soluzione efficace basata su una semplice convenzione:
 - il nome del metodo ne definisce la semantica: la query viene costruita a partire dal nome del metodo (senza scrivere codice jpql)
 - l'idea è di specificare nel nome del metodo:
 - l'operazione che si vuol eseguire (find, delete, count, exists)
 - quale combinazione di proprietà (attributi) deve essere verificata nei valori restituiti (ad es. cognome = ? AND nome = ?)
 - i parametri del metodo costituiscono i parametri della query

CrudRepository: Esempio

```
package it.uniroma3.siw.spring.repository;
```

```
import java.util.List;  
import org.springframework.data.repository CrudRepository;
```

```
import it.uniroma3.siw.spring.model.Movie;
```

```
public interface MovieRepository extends CrudRepository<Movie, Long> {
```

```
    public List<Movie> findByTitle(String title);
```

```
    public List<Movie> findByTitleAndAnno(String Titolo, Integer anno);
```

```
}
```

Select p
from Movie m where
title = :title

Select m
from Movie m
where title = :title or
anno = :anno

CrudRepository: costruzione di query

Per un elenco completo delle convenzioni che possono essere usate nella costruzione della query attraverso il nome dei metodi, vedi:

<https://docs.spring.io/spring-data/jpa/reference/repositories/query-keywords-reference.html>

CrudRepository: custom query

Attraverso la annotazione `@Query` è possibile associare un metodo ad una query custom, che può essere espressa

- In JPQL:

```
@Query("select m from Movie m where ...")  
List<Movie> findAllByStranaCondizione ();
```

- In SQL

```
Query("select ....", nativeQuery=true )  
List<Movie> findAllByStranaCondizione ();
```

Sommario

- Repository
- JPA CrudRepository
 - Il metodo save()
 - Costruzione di query
 - Custom query
- Eventi in cascata

Eventi in cascata

- Le operazioni di inserimento, cancellazione aggiornamento si realizzano con i repository
- Ci sono situazioni in cui vorremmo che alcune operazioni fossero propagate alle entità collegate attraverso associazioni

Esempio

- Siano date le classi Customer e Address viste negli esercizi sul mapping ORM
- Cosa ci aspettiamo da questa porzione di codice

```
customer.setFirstName("paolo");  
customer.setLastName("rossi");  
address.setStreet("vicolo cieco");  
address.setCity("roma");
```

```
customer.setAddress(address);
```

```
customerRepository.save(customer);
```

Esempio (cont.)

- Si verifica un errore perchè non abbiamo salvato l'oggetto Address prima di salvare l'oggetto Customer

```
customer.setFirstName("paolo");  
customer.setLastName("rossi");  
address.setStreet("vicolo cieco");  
address.setCity("roma");
```

```
customer.setAddress(address);
```

```
addressRepository.save(address);  
customerRepository.save(customer);
```

Eventi in cascata

- Sarebbe comodo che l'operazione svolta da **save()** sull'oggetto **Customer** si applicasse **in cascata** all'oggetto **Address** ad essa collegato

Eventi in cascata

- Si può ottenere l'applicazione di eventi in cascata (sulle entità collegate da associazioni) specificando questa proprietà nelle associazioni
- Sintatticamente si adorna l'annotazione dell'associazione con la coppia attributo valore
`cascade = {lista di eventi}`

Esempio

@Entity

```
public class Customer {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.AUTO)
```

```
    private Long id;
```

```
    @Column(nullable = false)
```

```
    private String firstName;
```

```
    @Column(nullable = false)
```

```
    private String lastName;
```

```
    @Column(unique=true, nullable = false)
```

```
    private String email;
```

```
    @OneToOne(cascade = {CascadeType.PERSIST, CascadeType.REMOVE})
```

```
    private Address address;
```

Eventi in Cascata

- Rivediamo l'esempio precedente. Ora questa porzione di codice funziona:

```
customer.setFirstName("paolo");  
customer.setLastName("rossi");  
address.setStreet("vicolo cieco");  
address.setCity("roma");
```

```
customer.setAddress(address);
```

```
customerRepository.save(customer);
```

Eventi in Cascata

CASCADE TYPE

- | | |
|-----------|--------------------|
| — PERSIST | inserimento |
| — REMOVE | cancellazione |
| — MERGE | aggiornamento |
| — REFRESH | |
| — CLEAR | |
| — ALL | tutti i precedenti |

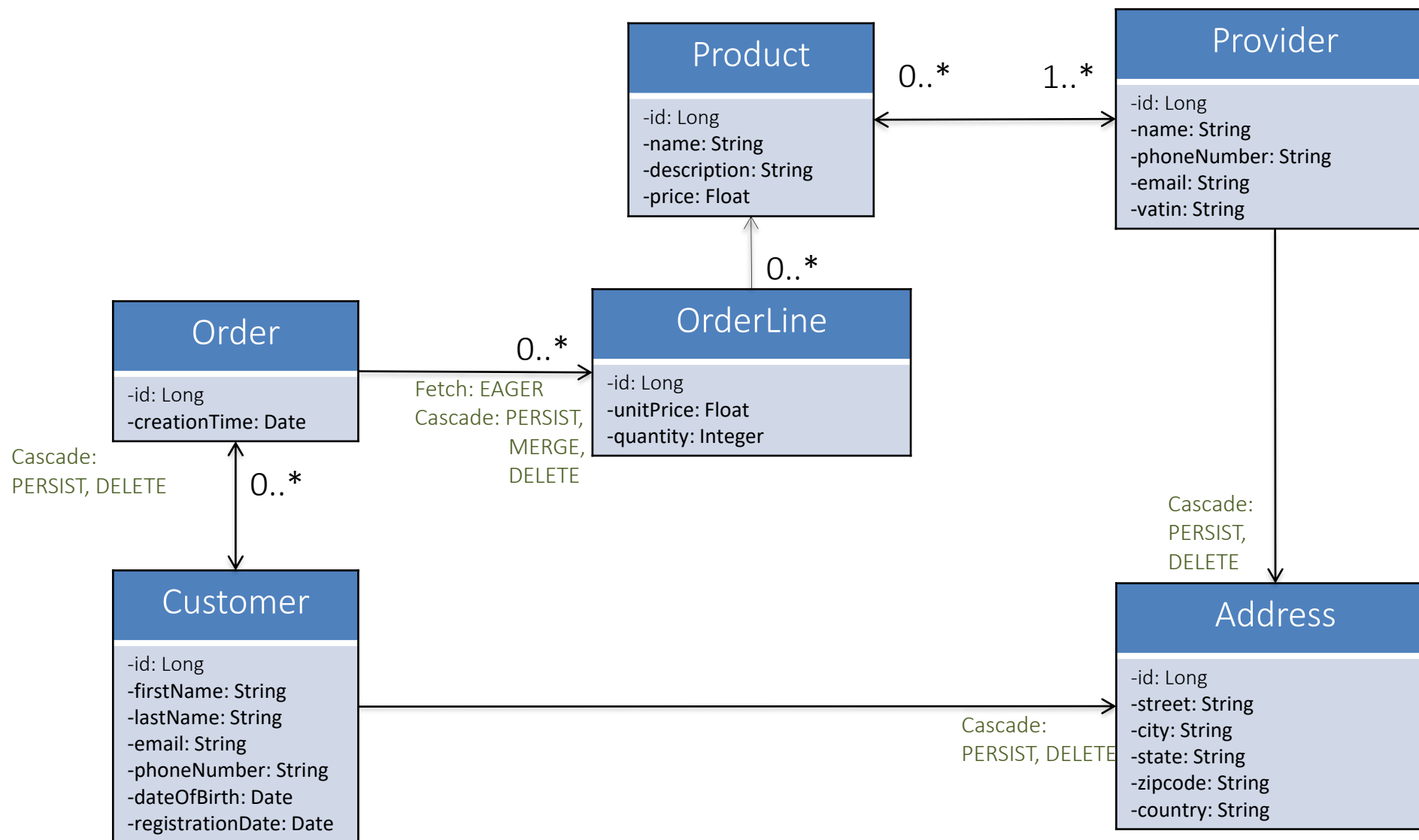
Eventi in Cascata

- Influiscono molto sulle prestazioni
 - Es.: fare il cascade del PERSIST di una relazione one-to-many comporta di verificare se gli oggetti della collezione sono già presenti nel database
- Una semplice linee guida:
 - Impostare cascade per PERSIST, MERGE solo su associazioni che esprimono una composizione (vedi APS)

Eventi in Cascata

- Come per le strategie di Fetch, anche gli eventi in cascata devono essere espressi in fase di progetto sul class diagram
- Anche in questo caso, adorniamo le associazioni con una nota

Esempio



Mettiamo tutto in pratica



Esercizio 6 (cont.)

- Modificare le classi dell'Esercizio 2 (slide 04) affinché siano rispettate le strategie di fetch e cascade riportati nel class diagram della slide precedente
- NB: è necessario ragionare sui casi d'uso. Fare assunzioni ragionevoli sulle operazioni offerte dal sistema
- Scrivere una classe di test per verificare il comportamento degli eventi in cascata